

ĐẠI HỌC BÁCH KHOA HÀ NỘI



BÁO CÁO BÀI TẬP VỀ NHÀ 04

Sinh viên thực hiện: Ngô Đức Anh

Giảng viên hướng dẫn: Thầy Lê Bá Vui

Bộ môn: Lập trình mạng

Trường: Công nghệ thông tin & truyền thông

HÀ NỘI, 4/2026

1. Bài tập 1: Chat Server

1.1. Yêu cầu

Sử dụng hàm `select()/poll()`, viết chương trình **chat_server** thực hiện các chức năng sau:

Nhận kết nối từ các client, và vào hỏi tên client cho đến khi client gửi đúng cú pháp:

“client_id: client_name”

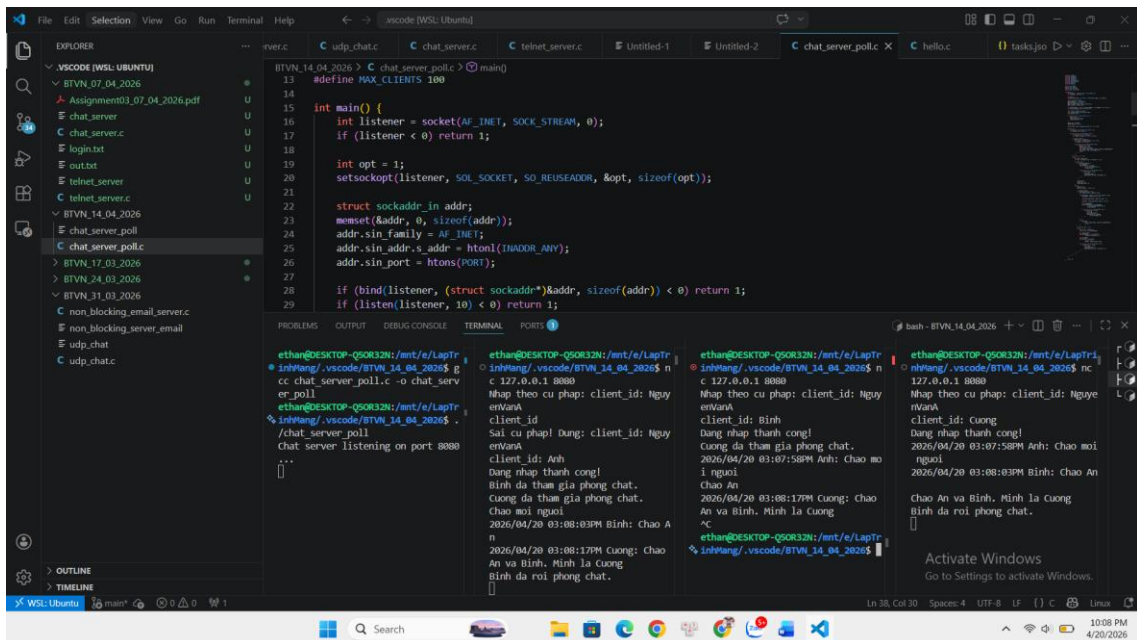
trong đó `client_name` là tên của client, xâu ký tự viết liền. Sau đó nhận dữ liệu từ một client và gửi dữ liệu đó đến các client còn lại, ví dụ: client có id “abc” gửi “xin chào” thì các client khác sẽ nhận được: “abc: xin chào” hoặc có thể thêm thời gian vào trước ví dụ: “2023/05/06 11:00:00PM abc: xin chào”.

1.2. Mô tả bài toán:

Bài toán yêu cầu xây dựng một chương trình **máy chủ trò chuyện (Chat Server)** sử dụng giao thức **TCP Socket** trên hệ điều hành Linux. Chương trình cho phép nhiều máy khách (client) kết nối đồng thời đến server và trao đổi tin nhắn trong cùng một phòng chat.

Server phải sử dụng cơ chế **I/O Multiplexing** thông qua hàm `poll()` để quản lý nhiều kết nối cùng lúc mà không cần tạo nhiều tiến trình hoặc nhiều luồng xử lý riêng biệt.

1.3. Kết quả thực nghiệm



Hình 1. Demo bài toán

- 1) Ta sẽ minh họa chương trình trên bằng cách kết nối 3 client gồm AN, BINH, Cuong và 1 server.
- 2) Chạy chương trình `chat_server` thì server hiện thị “Chat server listening on port 8080...”

- 3) Chạy netcat bằng cách thực hiện lệnh `nc 127.0.0.1 8080`, sau khi kết nối tới server thì hiển thị thông báo “Nhập theo cú pháp: `client_id: NguyenVanA`”.
- 4) Chương trình sẽ hiển thị thông báo lỗi nếu cú pháp đăng nhập không đúng. Nếu đăng nhập thành công thì bên client khác sẽ nhận được thông báo đã gia nhập phòng chat.
- 5) Nếu kết thúc chương trình ở client thì sẽ hiển thị thông báo đã rời khỏi phòng chat ở client khác.

2. Bài tập 2: Telnet Server

2.1. Yêu cầu

Sử dụng hàm `select()/poll()`, viết chương trình `telnet_server` thực hiện các chức năng sau:

Khi đã kết nối với 1 client nào đó, yêu cầu client gửi `user` và `pass`, so sánh với file cơ sở dữ liệu là một file text, mỗi dòng chứa một cặp `user + pass` ví dụ:

`admin admin`

`guest nopass`

...

Nếu so sánh sai, không tìm thấy tài khoản thì báo lỗi đăng nhập.

Nếu đúng thì đợi lệnh từ client, thực hiện lệnh và trả kết quả cho client.

Dùng hàm `system(“dir > out.txt”)` để thực hiện lệnh.

`dir` là ví dụ lệnh `dir` mà client gửi. `> out.txt` để định hướng lại dữ liệu ra từ lệnh `dir`, khi đó kết quả lệnh `dir` sẽ được ghi vào file văn bản.

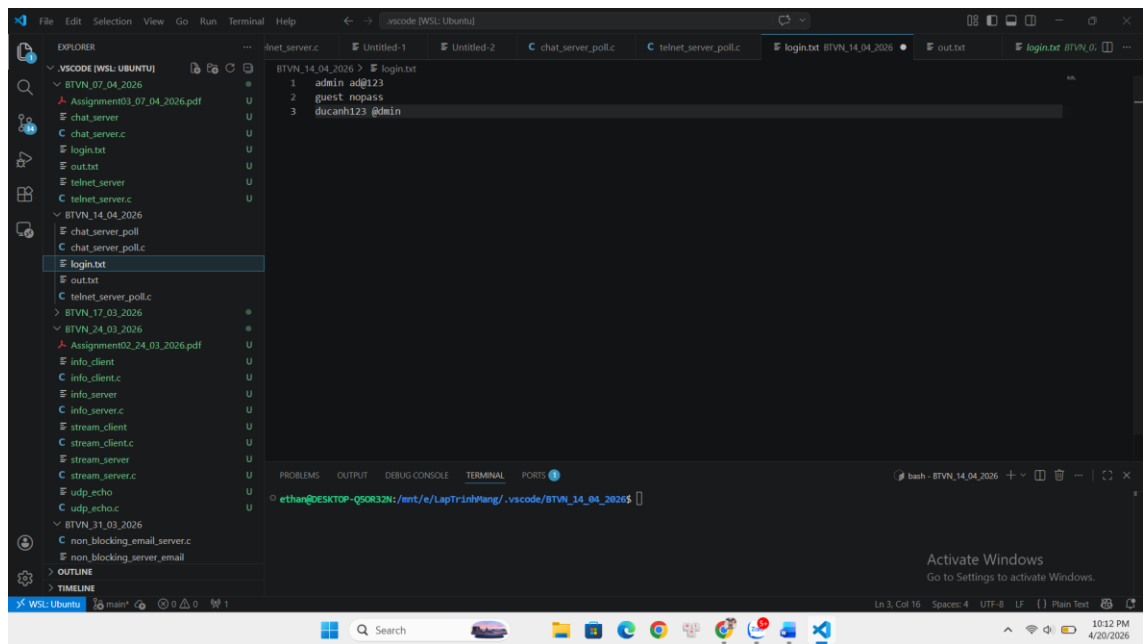
2.2. Mô tả bài toán

Bài toán yêu cầu xây dựng một chương trình **Telnet Server** hoạt động trên nền giao thức **TCP** bằng ngôn ngữ **C** trong môi trường **Linux**. Chương trình cho phép nhiều máy khách (client) kết nối đồng thời đến server thông qua mạng và đăng nhập bằng tài khoản được lưu trong file `login.txt`.

Server sử dụng hàm **`pool()`** để giám sát nhiều socket cùng lúc, nhờ đó có thể phục vụ nhiều client mà không cần tạo nhiều tiến trình hoặc nhiều luồng riêng.

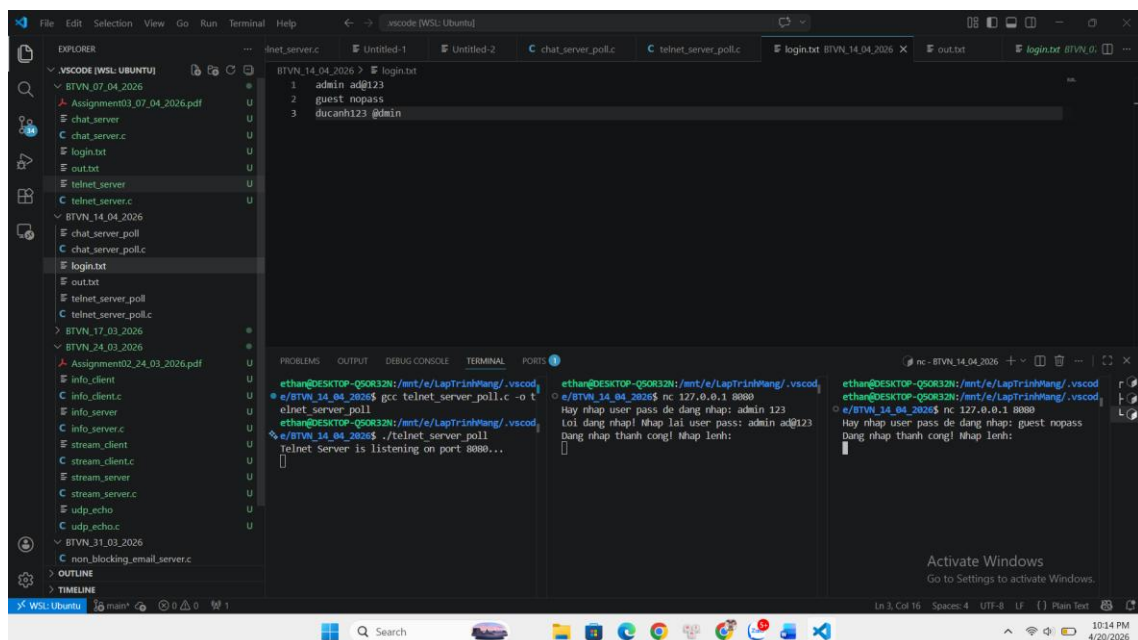
2.3. Kết quả thực nghiệm

- 1) Nội dung file `login.txt` mà chương trình sử dụng để tham chiếu khi client có thực hiện yêu cầu đăng nhập



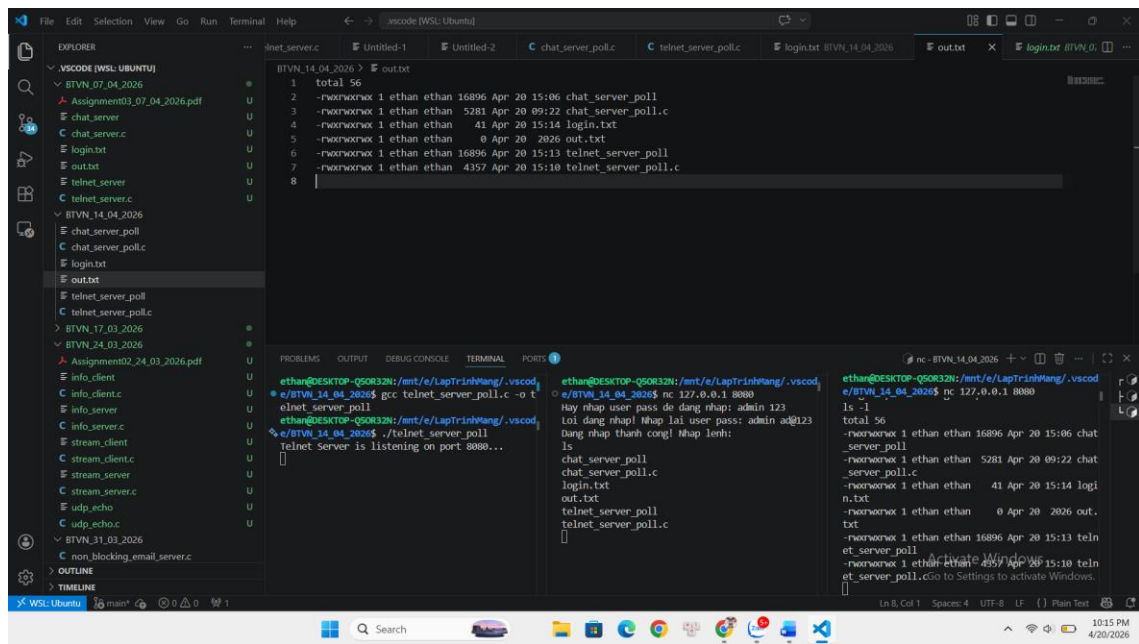
Hình 2. Nội dung file login.txt

- 2) Ta sẽ minh họa trên 3 terminal gồm 1 server và 2 client là admin và guest
- 3) Chạy chương trình telnet_server. Kết quả hiển thị “Chat server listening on port 8080...”
- 4) Chạy netcat bằng cách thực hiện lệnh nc 127.0.0.1 8080, sau khi kết nối tới server thì hiển thị thông báo “Hay nhập user pass de dang nhap”
 - Trong trường hợp nhập sai thì client yêu cầu nhập lại. Nếu đúng thì hiển thị đăng nhập thành công (Xem hình 3.)



Hình 3. Demo đăng nhập

- Khi nhập lệnh ls -l phía admin thì file output.txt sẽ hiển thị nội dung của lệnh thực hiện terminal



Hình 4. Demo lệnh `ls -l` và file `output.txt`